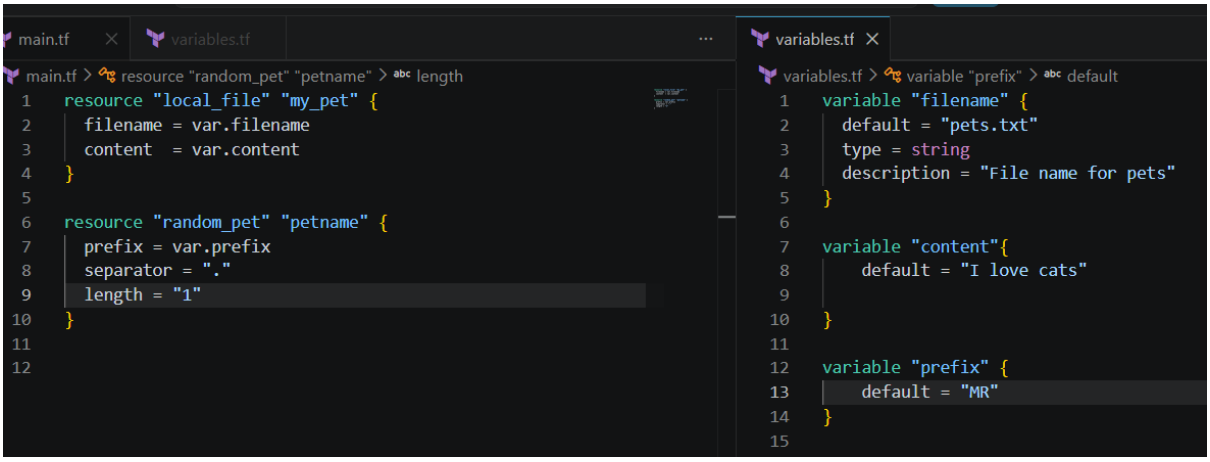
Terraform-03

1. **Watch the Terraform-03 video.**

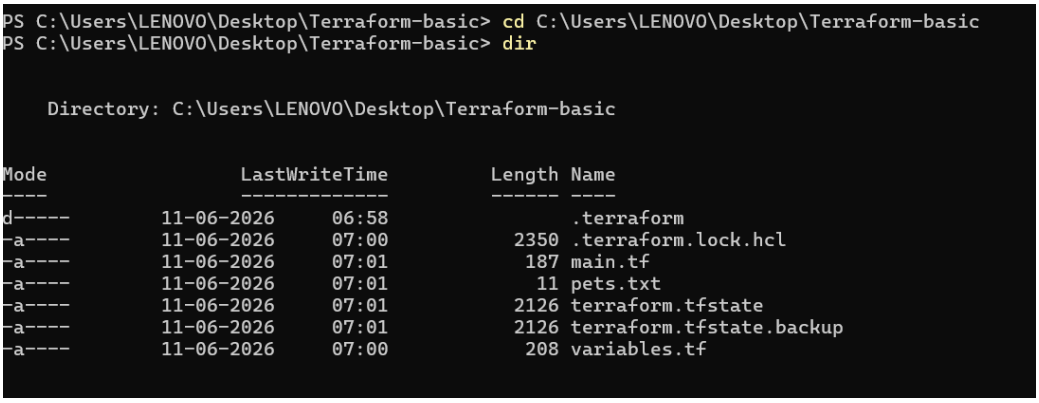
   - Using of variables: By using varibles.tf file



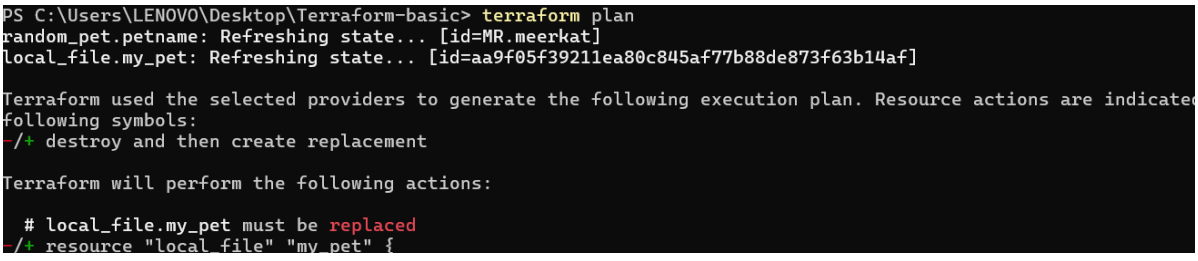
```
main.tf > resource "random_pet" "petname" > length
1  resource "local_file" "my_pet" {
2    filename = var.filename
3    content  = var.content
4  }
5
6  resource "random_pet" "petname" {
7    prefix = var.prefix
8    separator = "."
9    length = "1"
10 }
11
12
```

```
variables.tf > variable "prefix" > default
1  variable "filename" {
2    default = "pets.txt"
3    type = string
4    description = "File name for pets"
5  }
6
7  variable "content"{
8      default = "I love cats"
9
10 }
11
12 variable "prefix" {
13     default = "MR"
14 }
15
```

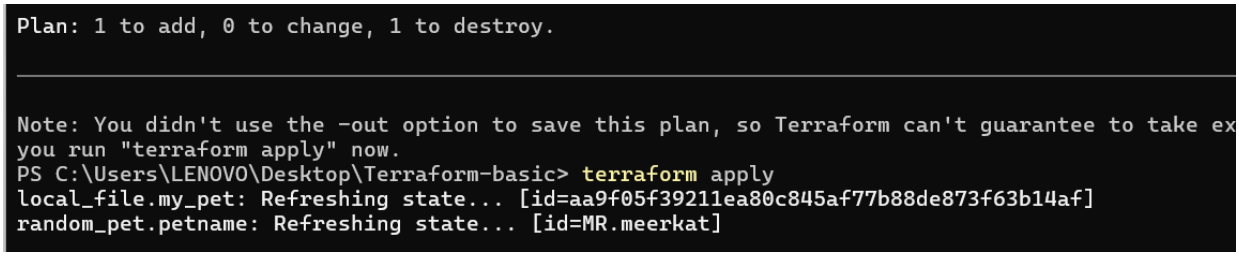
   - Create diR wild.txt: By using interactive mode



```
PS C:\Users\LENOVO\Desktop\Terraform-basic> cd C:\Users\LENOVO\Desktop\Terraform-basic
PS C:\Users\LENOVO\Desktop\Terraform-basic> dir


    Directory: C:\Users\LENOVO\Desktop\Terraform-basic


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        11-06-2026     06:58                .terraform
-a----        11-06-2026     07:00           2350 .terraform.lock.hcl
-a----        11-06-2026     07:01            187 main.tf
-a----        11-06-2026     07:01             11 pets.txt
-a----        11-06-2026     07:01           2126 terraform.tfstate
-a----        11-06-2026     07:01           2126 terraform.tfstate.backup
-a----        11-06-2026     07:00            208 variables.tf
```



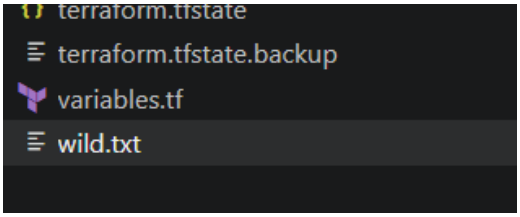
```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform plan
random_pet.petname: Refreshing state... [id=MR.meerkat]
local_file.my_pet: Refreshing state... [id=aa9f05f39211ea80c845af77b88de873f63b14af]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicate
following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

  # local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
```

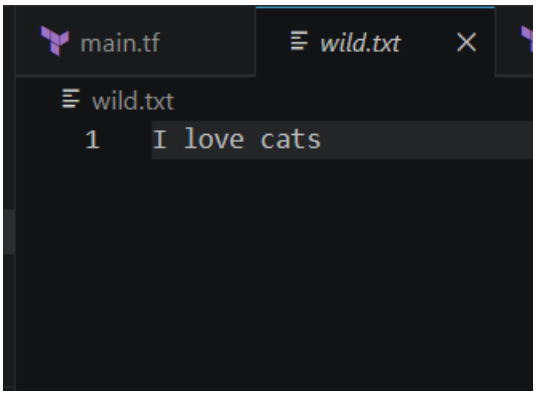


```
Plan: 1 to add, 0 to change, 1 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take ex
you run "terraform apply" now.
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
local_file.my_pet: Refreshing state... [id=aa9f05f39211ea80c845af77b88de873f63b14af]
random_pet.petname: Refreshing state... [id=MR.meerkat]
```

```
  Enter a value: yes

local_file.my_pet: Destroying... [id=aa9f05f39211ea80c845af77b88de873f63b14af]
local_file.my_pet: Destruction complete after 0s
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=aa9f05f39211ea80c845af77b88de873f63b14af]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
```

After terraform init, plan and apply then we get new wild.txt





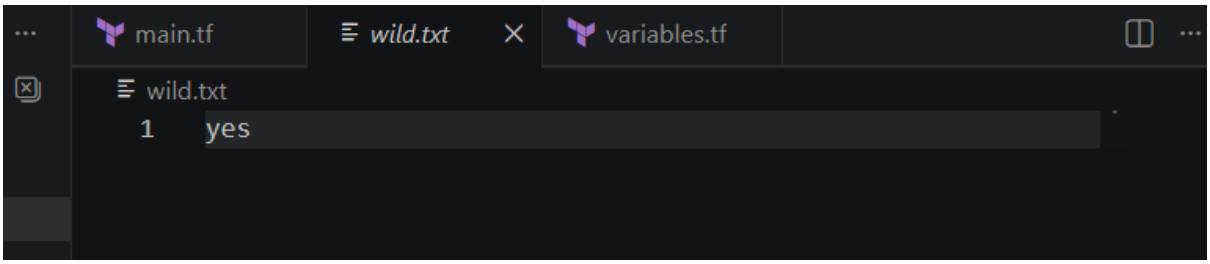
- Command line flags


```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
var.content
  Enter a value: yes

var.prefix
  Enter a value: no

random_pet.petname: Refreshing state... [id=MR.meerkat]
local_file.my_pet: Refreshing state... [id=aa9f05f39211ea80c845af77b88de873f63b14af]
```

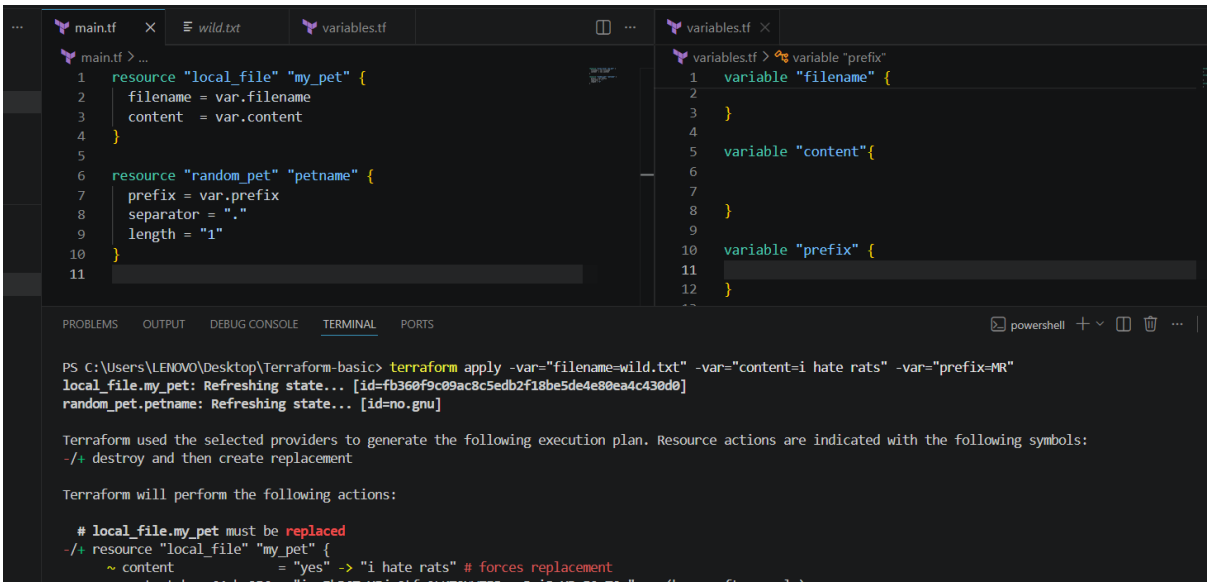
```
  # random_pet.petname must be replaced
-/+ resource "random_pet" "petname" {
      ~ id        = "MR.meerkat" -> (known after apply)
      ~ prefix    = "MR" -> "no" # forces replacement
        # (2 unchanged attributes hidden)
    }
```

```
Plan: 2 to add, 0 to change, 2 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes
```

```
random_pet.petname: Creating...
local_file.my_pet: Creating...
random_pet.petname: Creation complete after 0s [id=no.gnu]
local_file.my_pet: Creation complete after 0s [id=fb360f9c09ac8c5edb2f18be5de4e80ea4c430d0]

Apply complete! Resources: 2 added, 0 changed, 2 destroyed.
PS C:\Users\LENOVO\Desktop\Terraform-basic>
```



Command line flags: terraform apply - var "filename=/root/pets.txt" -var "prefix=MR"



-

main.tf | wild.txt | variables.tf

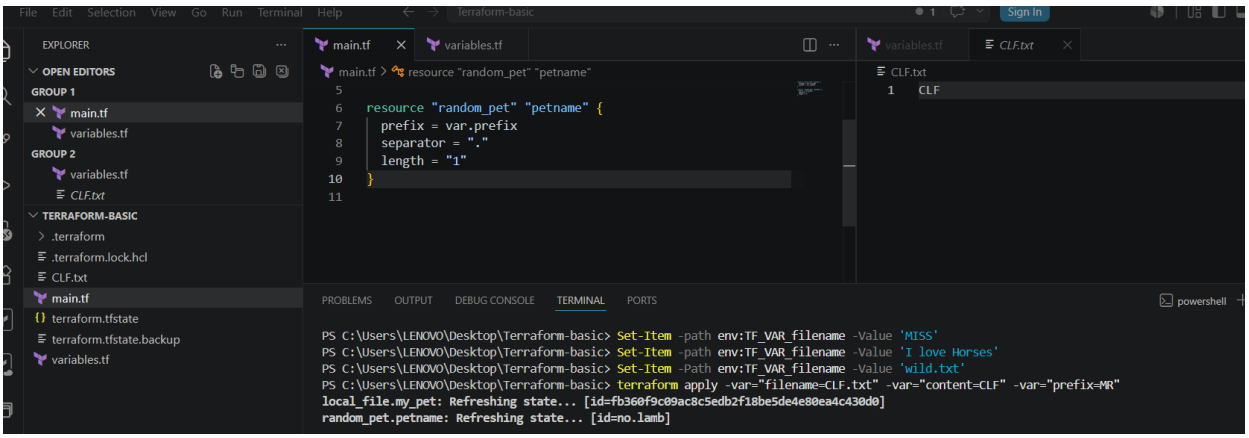
wild.txt

```
1   i hate rats
```

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> Set-Item -path env:TF_VAR_filename -Value 'wild.txt'
PS C:\Users\LENOVO\Desktop\Terraform-basic> Set-Item -path env:TF_VAR_filename -Value 'I love Horses'
PS C:\Users\LENOVO\Desktop\Terraform-basic> Set-Item -path env:TF_VAR_filename -Value 'MISS'
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
var.content
  Enter a value: Horses

var.prefix
  Enter a value: MISS

random_pet.petname: Refreshing state... [id=No.woodcock]
local_file.my_pet: Refreshing state... [id=fb360f9c09ac8c5edb2f18be5de4e80ea4c430d0]
```

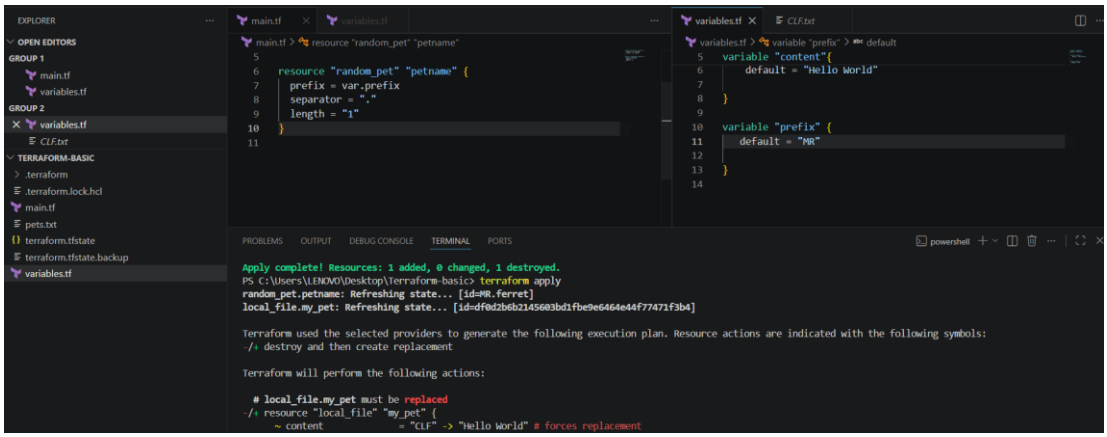
```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

random_pet.petname: Destroying... [id=No.woodcock]
local_file.my_pet: Destroying... [id=fb360f9c09ac8c5edb2f18be5de4e80ea4c430d0]
random_pet.petname: Destruction complete after 0s
local_file.my_pet: Destruction complete after 0s
random_pet.petname: Creating...
random_pet.petname: Creation complete after 0s [id=MISS.terrapin]
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=29ed16888c8beb19c7a0740a95b969d2562314a4]

Apply complete! Resources: 2 added, 0 changed, 2 destroyed.
```

  - ➢ Environment variables
    export TF_VAR_filename="/root.pets.txt"
    export TF_VAR_prefix= "MR"
    Set-Item -Path env:TF_VAR_filename -Value 'wild.txt



```
Plan: 2 to add, 0 to change, 2 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.my_pet: Destroying... [id=fb360f9c09ac8c5edb2f18be5de4e80ea4c430d0]
random_pet.petname: Destroying... [id=no.lamb]
random_pet.petname: Destruction complete after 0s
local_file.my_pet: Destruction complete after 0s
random_pet.petname: Creating...
random_pet.petname: Creation complete after 0s [id=MR.escargot]
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=df0d2b6b2145603bd1fbe9e6464e44f77471

Apply complete! Resources: 2 added, 0 changed, 2 destroyed.
```

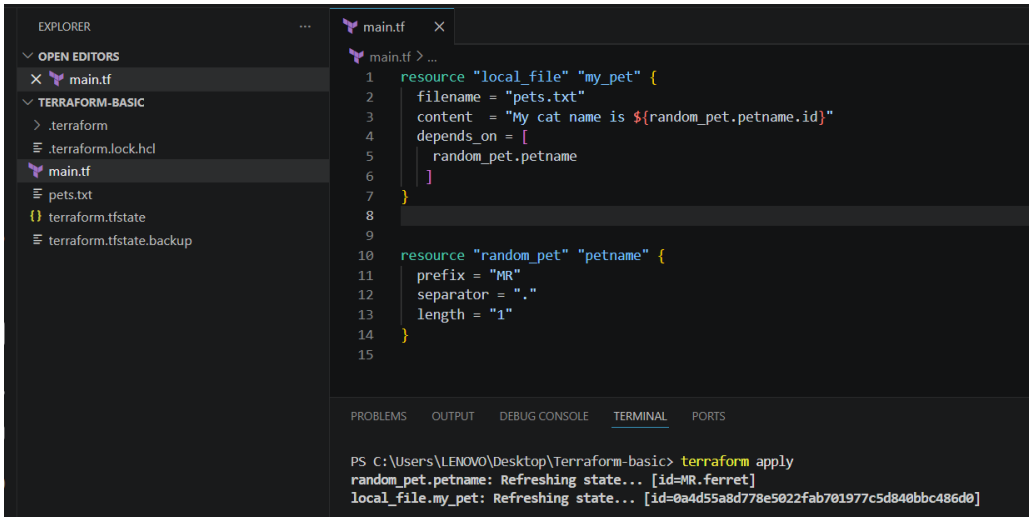
- Terraform apply next




```
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.my_pet: Destroying... [id=df0d2b6b2145603bd1fbe9e6464e44f77471f3b4]
local_file.my_pet: Destruction complete after 0s
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=0a4d55a8d778e5022fab701977c5d840bbc486d0]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
PS C:\Users\LENOVO\Desktop\Terraform-basic>
```

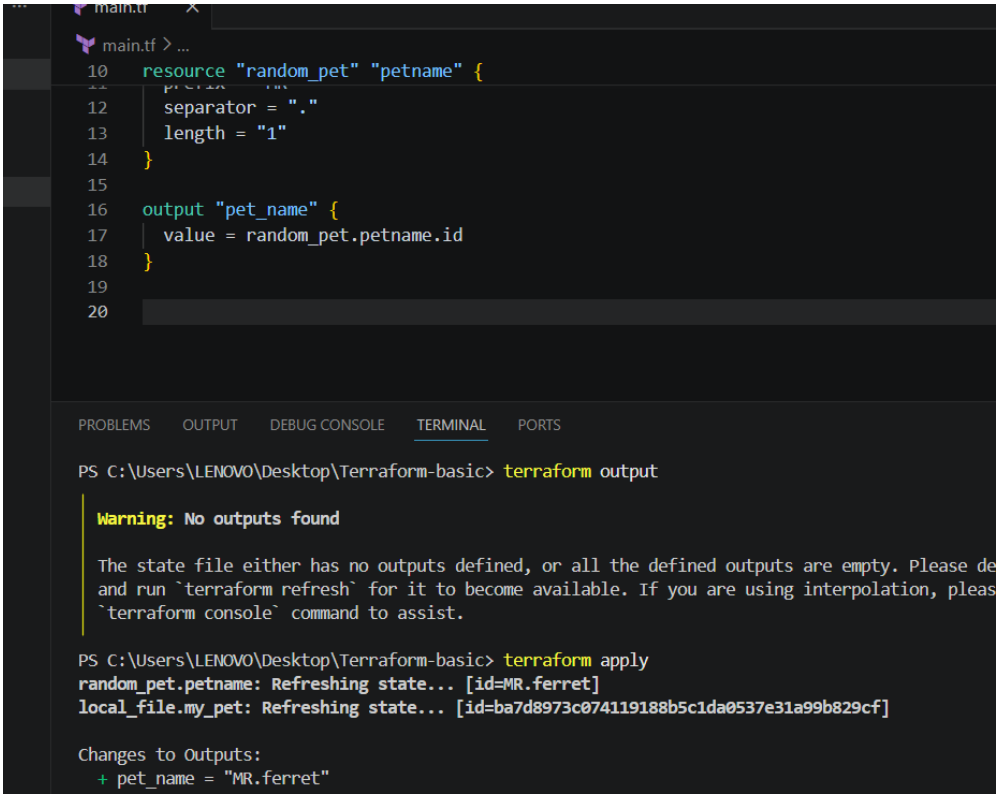
2. Execute the script shown in the video.

- Close vs code and open it again so environment variables close and apply now



```
Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.my_pet: Destroying... [id=0a4d55a8d778e5022fab701977c5d840bbc486d0]
local_file.my_pet: Destruction complete after 0s
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=ba7d8973c074119188b5c1da0537e31a99b829cf

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
PS C:\Users\LENOVO\Desktop\Terraform-basic>
```

- Now check output of terraform after apply




```
  Enter a value: yes


Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

pet_name = "MR.ferret"
PS C:\Users\LENOVO\Desktop\Terraform-basic>
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform output
pet_name = "MR.ferret"
PS C:\Users\LENOVO\Desktop\Terraform-basic>
```

- Terraform show

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform output
pet_name = "MR.ferret"
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform show
# local_file.my_pet:
resource "local_file" "my_pet" {
    content              = "My cat name is MR.ferret"
    content_base64sha256 = "wwFqfFNOOLl0M0eRd77MAmjQS9BoytV9OQxNTQlEJro="
    content_base64sha512 = "DjX1NG9CEMuwJRroC0SfzWW04gWL8yf7toRu/jY7qK1IiiQ5mU5iccVy/kndIKKJwbZ9PeT89E
    content_md5          = "0b3916d81ff041be1b82180596d0f99c"
    content_sha1         = "ba7d8973c074119188b5c1da0537e31a99b829cf"
    content_sha256       = "c3016a7c534e38b97433479177becc0268d04bd068cad57d390c4d4d094426ba"
    content_sha512       = "0e35f5346f4210cbb0251ae80b449fcd65b4e2058bf327fbb6846efe363ba8ad488a243999
    directory_permission = "0777"
    file_permission      = "0777"
    filename             = "pets.txt"
    id                   = "ba7d8973c074119188b5c1da0537e31a99b829cf"
```

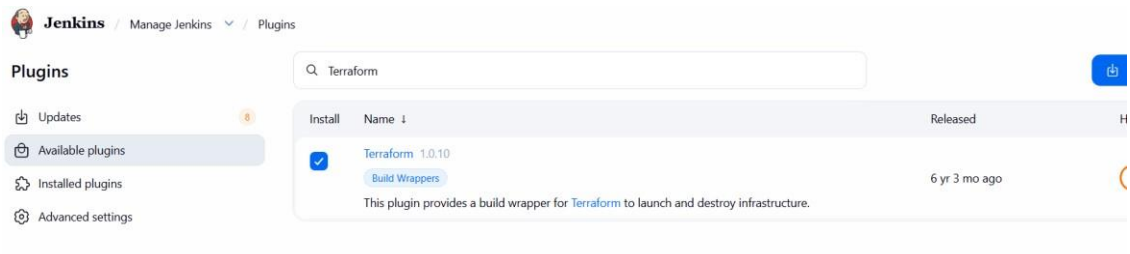
3. Integrate Terraform in Jenkins using the Terraform plugin.

Open Jenkins server and check running or not and version also

```
root@ip-172-31-22-87:~# systemctl start jenkins
root@ip-172-31-22-87:~# systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
     Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-06-09 17:22:57 UTC; 37s ago
   Main PID: 519 (java)
      Tasks: 45 (limit: 1013)
     Memory: 389.3M (peak: 390.6M)
        CPU: 22.347s
     CGroup: /system.slice/jenkins.service
             └─519 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.wa
```

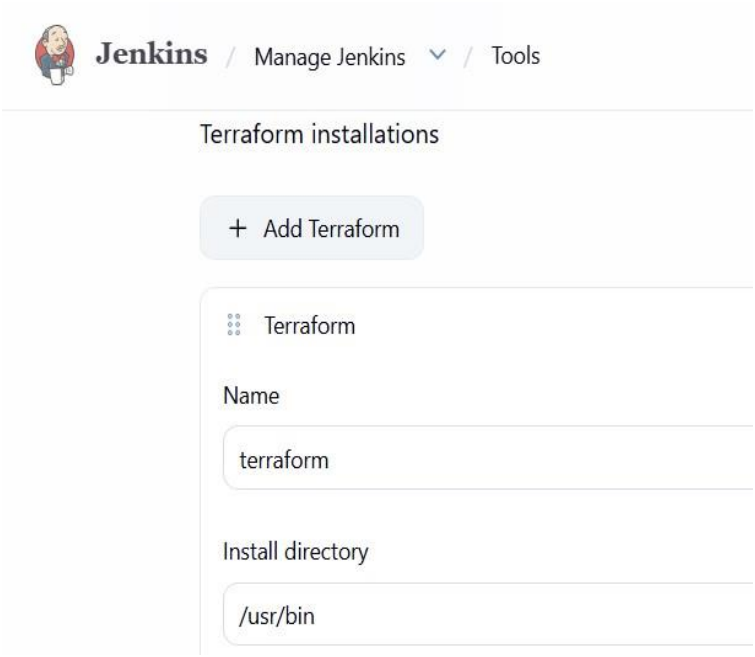
```
root@ip-172-31-22-87:~# terraform -v
Terraform v1.15.5
on linux_amd64
root@ip-172-31-22-87:~#
```

Open Jenkins in browser and install Terraform plugin

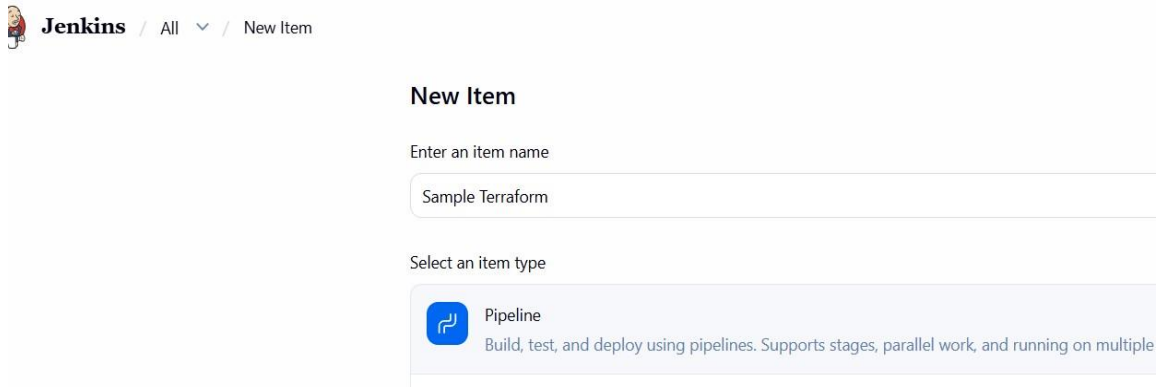


In Jenkins manage select Tools and add Terraform installations details

-



Now select new job item and select pipeline



Now in Jenkins server go to workspace of new created job and there create main.tf

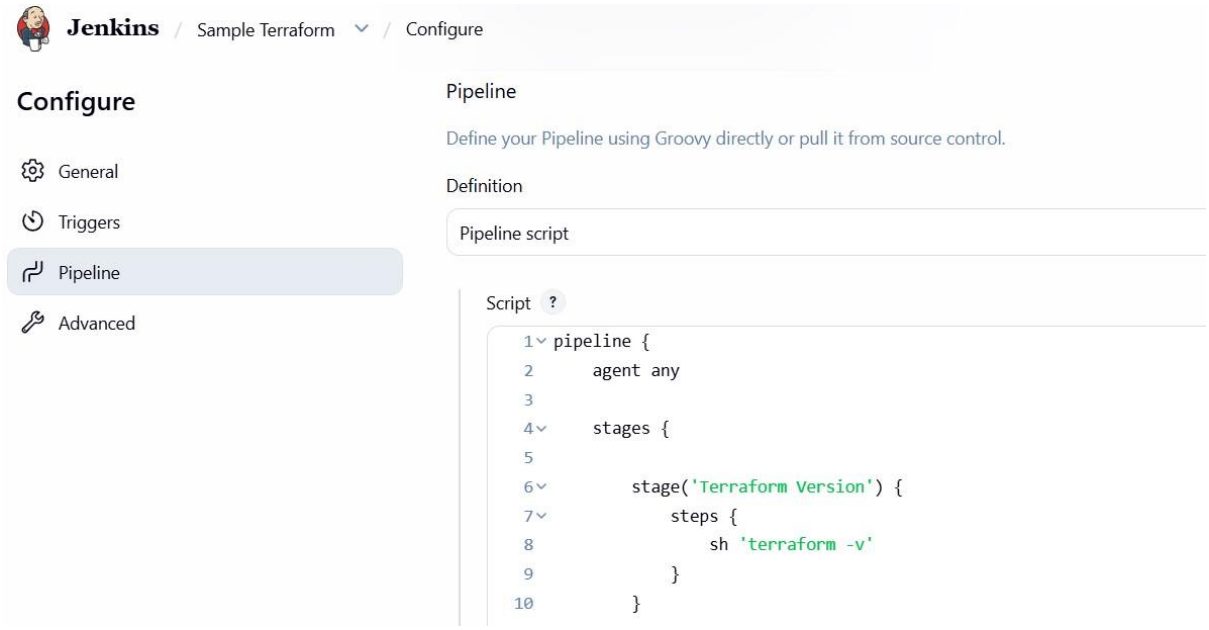
```
root@ip-172-31-22-87:/var/lib/jenkins/workspace# cd ..
root@ip-172-31-22-87:/var/lib/jenkins# cd workspace/Sample\ Terraform
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Sample Terraform# vi main.tf
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Sample Terraform# ls
main.tf
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Sample Terraform#
```

Now in main.tf file write script and save it

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

resource "local_file" "demo" {
  filename = "terraform-from-jenkins.txt"
  content  = "Terraform executed successfully from Jenkins using Terraform Plugin"
}
~
```

-

Now Pipeline and write a declarative script for the new created job

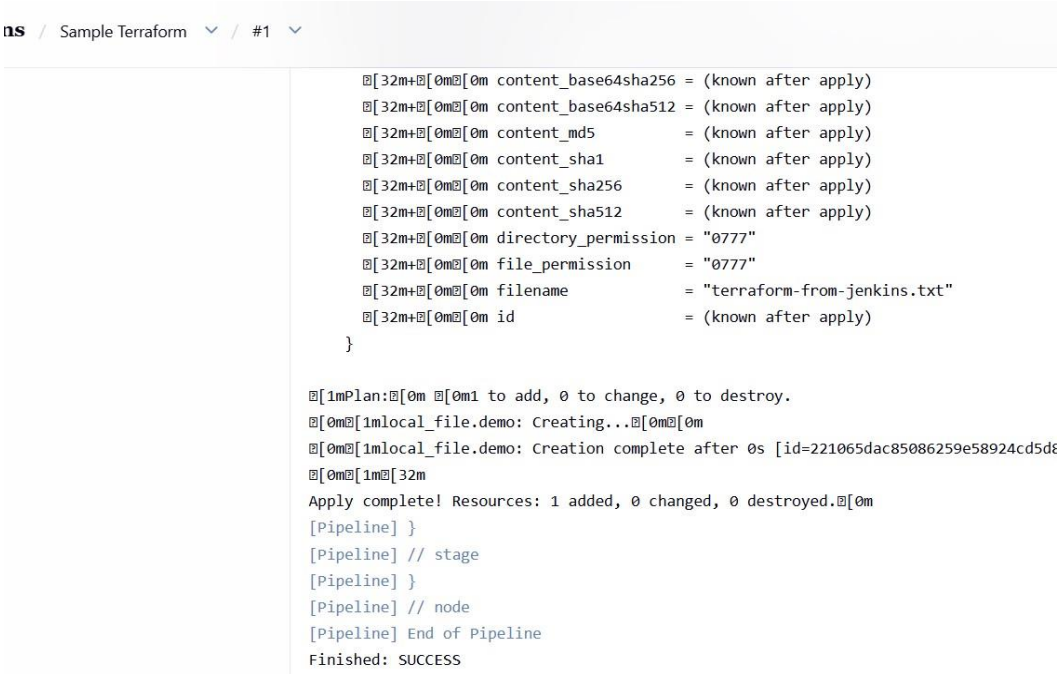


Pipeline Script:

```
pipeline {
   agent any

   stages {

      stage('Terraform Version') {
         steps {
            sh 'terraform -v'
         }
      }
      stage('Terraform
Init') {          steps {
            sh 'terraform init'
}

      stage('Terraform Plan') {
         steps {
            sh 'terraform plan'
         }
      }
      stage('Terraform Apply') {
         steps {
            sh 'terraform apply -auto-approve'
         }
      }
   }
}
```

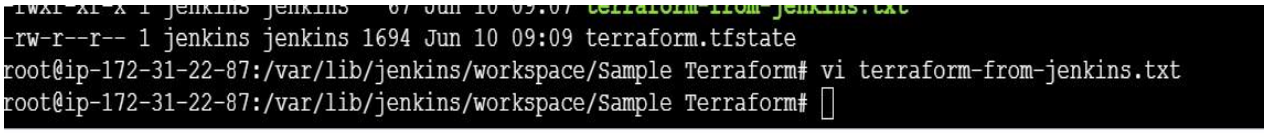
-

Successful script



Now check in server successful

To check open Vi terraform-from-jenkins.txt

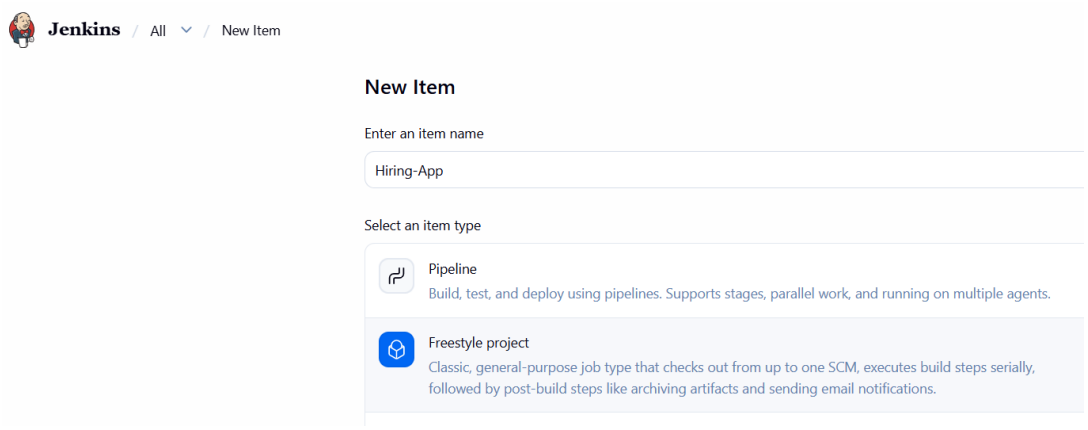


Successfully from Jenkins using Terraform plugin

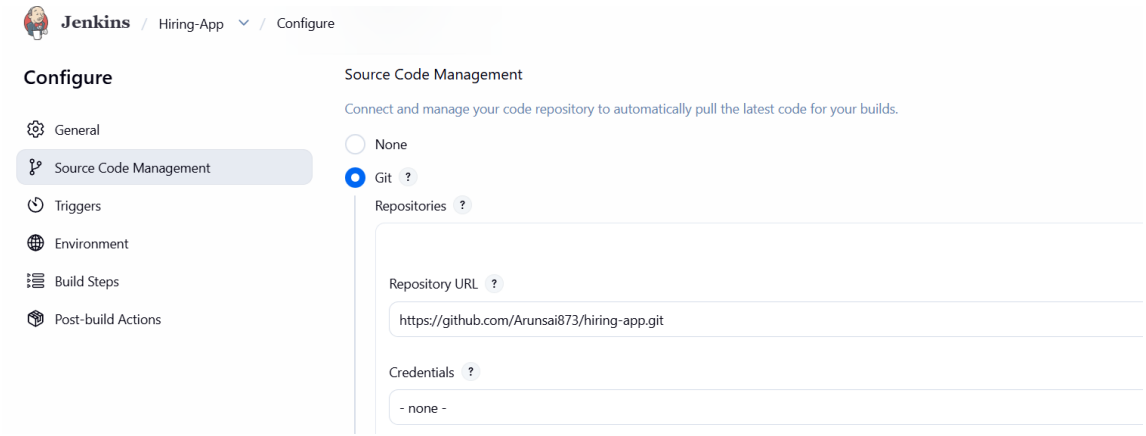
4. Create one Jenkins job using **Maven Project** for the code below with **two stages**:
    - **Stage 1:** Git clone
    - **Stage 2:** Maven Compilation
    - **Code:** https://github.com/betawins/java-Working-app.git

➢ Connect with Jenkins server and open in browser and create a new job

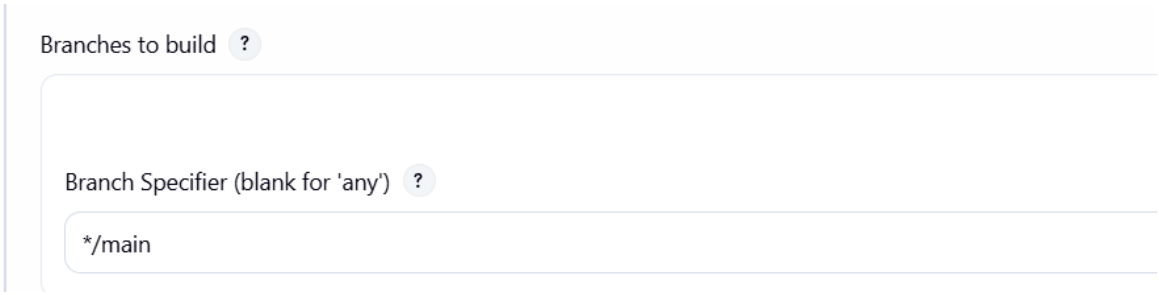
-



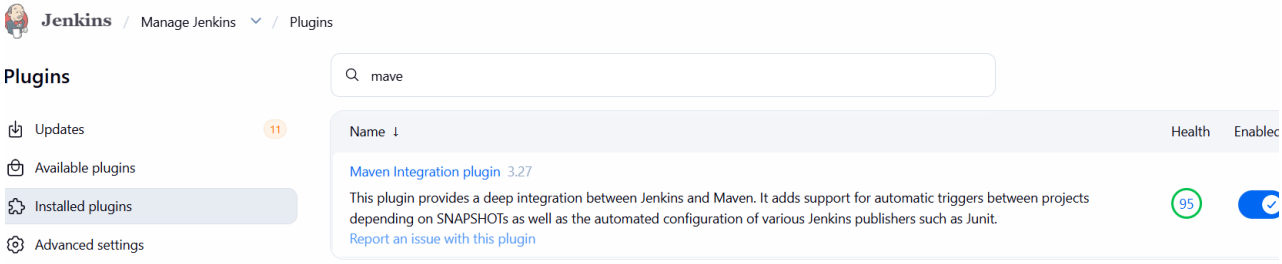
➢ Select Source code as git and cloning URL id paste it



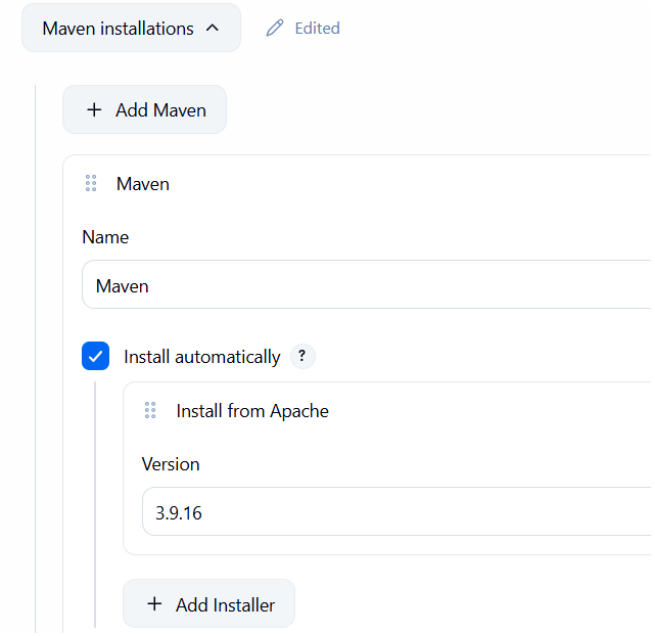
➢ Branch set as main and save it



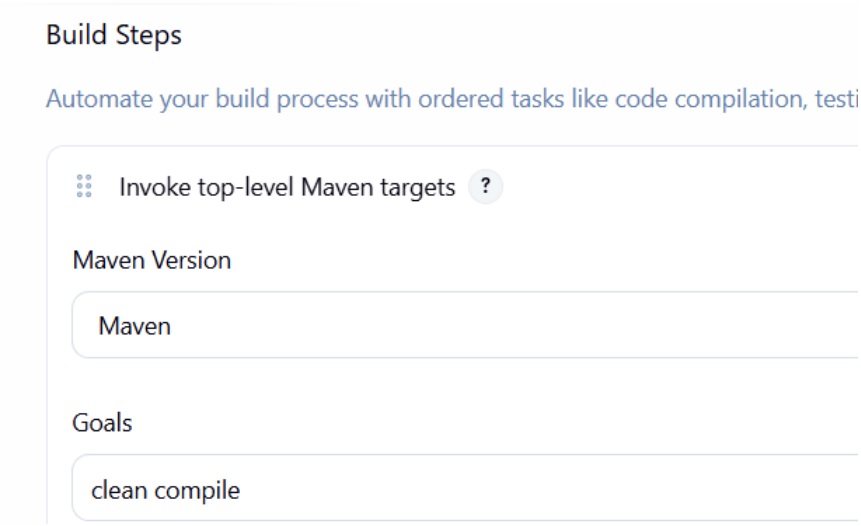
➢ Maven integration plugin installed



- In Manage Jenkins open and in tools select maven and select instant automatically



- In Build steps select maven and set goals and save it and build now

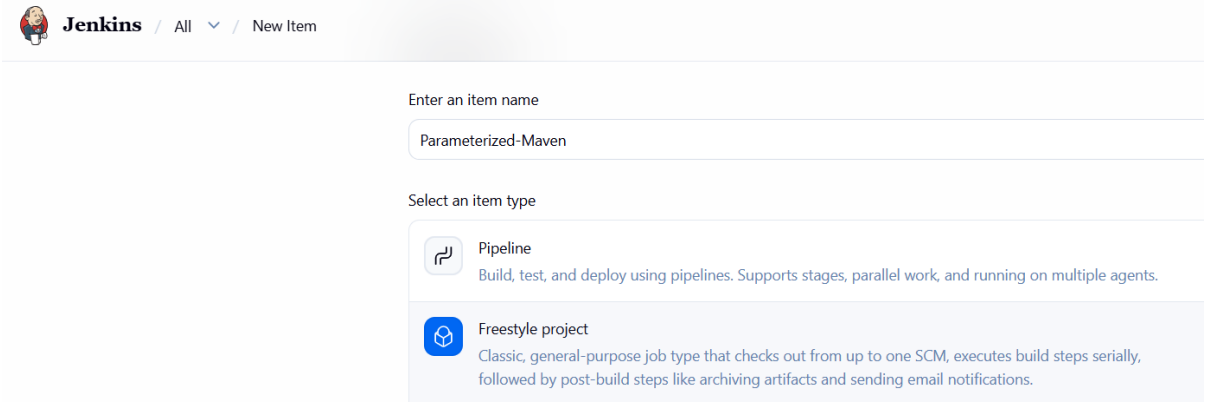


- Successfully Build

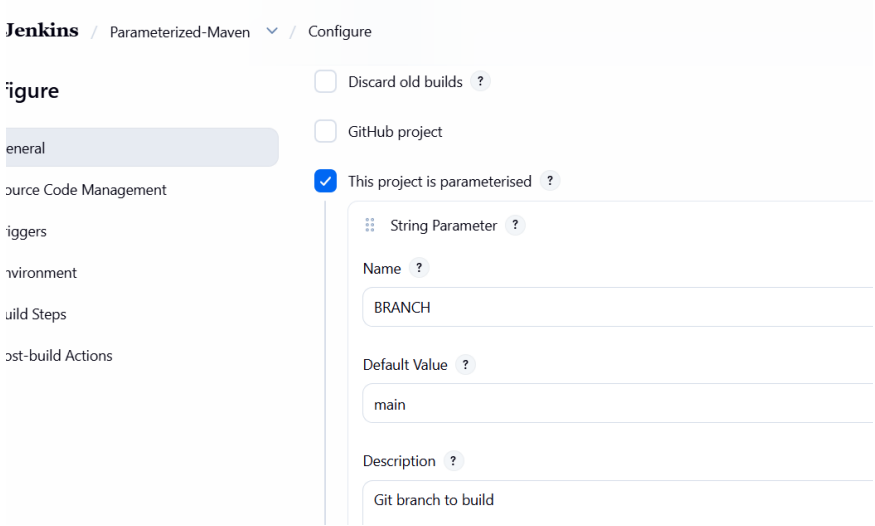
```
Progress (1): 11/30 kB
Progress (1): 28/30 kB
Progress (1): 30 kB

Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-c
2.16.2.jar (30 kB at 490 kB/s)
[INFO] No sources to compile
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  5.035 s
[INFO] Finished at: 2026-06-11T09:54:28Z
[INFO] ------------------------------------------------------------------------
Finished: SUCCESS
```

5. Use the same code and create a **parameterized job** in Jenkins with:
    - **Stage 1:** Git clone
    - **Stage 2:** Maven Compilation
    - **Code:** https://github.com/betawins/java-Working-app.git

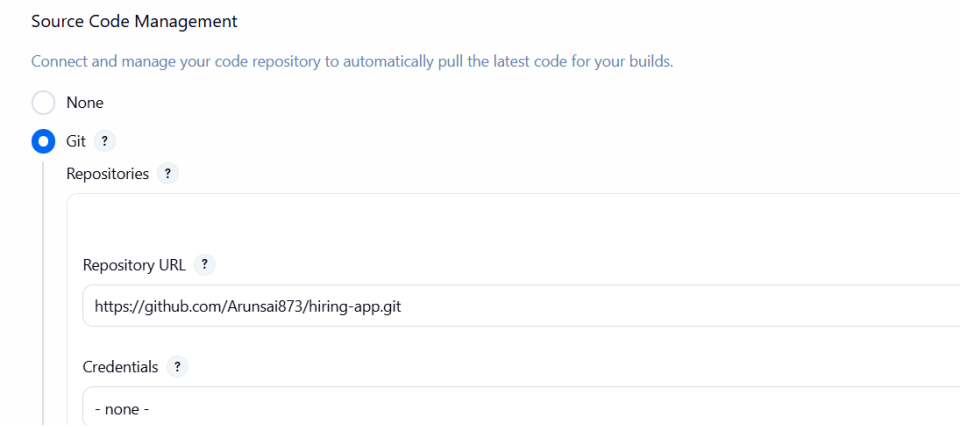
➢ Open Jenkins browsers and select new job and create with freestyle



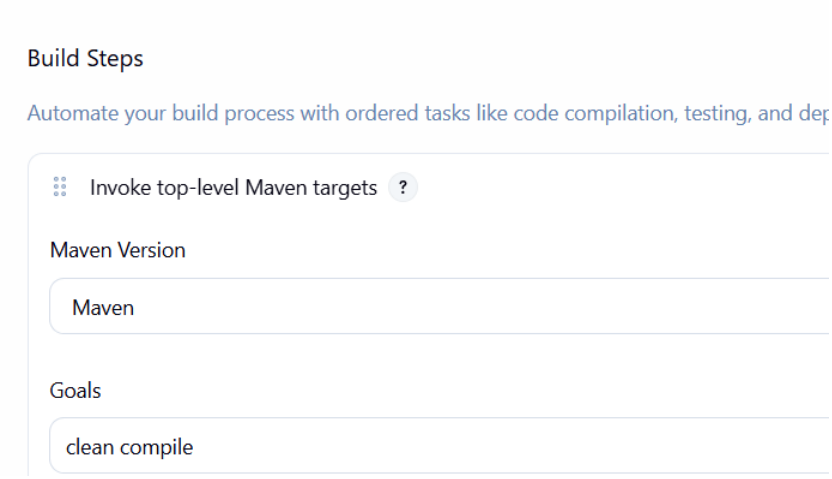
➢ Select Parameterized and select string parameter



➢ Select Git and given Git URL and select branch main



- Now select maven build and save it



- Successfully Build

```
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ hiring ---
[INFO] No sources to compile
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.343 s
[INFO] Finished at: 2026-06-11T10:07:40Z
[INFO] ------------------------------------------------------------------------
Finished: SUCCESS
```

6. What are the **global variables** in Jenkins?

Global variables in Jenkins are predefined environment variables accessible across jobs and pipelines. They provide build-related information such as build number, workspace path, job name, Git branch, and Jenkins home directory.
Some commonly used global variables include
BUILD_NUMBER, which stores the current build number,
JOB_NAME, which contains the name of the Jenkins job, and
WORKSPACE, which defines the location where Jenkins clones the source code and executes builds.

Variables like GIT_COMMIT and GIT_BRANCH help identify the exact Git commit and branch used during the build process.
These global variables are useful in automation, scripting, and CI/CD pipelines to make Jenkins jobs more dynamic and reusable.

-

Common Global Variables

Variable	Purpose
BUILD_NUMBER	Current build number
BUILD_ID	Build ID
JOB_NAME	Name of Jenkins job
BUILD_URL	URL of current build
WORKSPACE	Workspace location
JENKINS_URL	Jenkins server URL
NODE_NAME	Jenkins agent name
BUILD_TAG	Unique build tag
GIT_BRANCH	Current Git branch
GIT_COMMIT	Commit ID
EXECUTOR_NUMBER	Executor number
HOME	Jenkins home directory